Video Voiceover Script

Forecasting Daily Equity Volatility: Can Machine Learning Beat Classical Models?
(Trimester 9 Term Project, 12 slides, about 10 minutes)

François Schmitt (Roll Number: 23035010523)
f.schmitt@op.iitg.ac.in

Recording reminders from the submission guide: keep your face visible throughout; state your name, programme and project verbally at the start (the slide-1 text below does this); the title slide must open the video and also serves as the YouTube thumbnail. Speak naturally; if you stumble, pause, correct yourself and carry on. Suggested screen shares: the consolidated notebook during slides 6 to 8, and the leaderboard output during slide 9.

Slide 1: Title (about 35 seconds)

Hello, my name is François Schmitt, roll number 23035010523. I am a student of the BSc Honours in Data Science and Artificial Intelligence, the online degree programme of IIT Guwahati, and this is my Trimester 9 term project, course DA 379. The project asks a simple question: when it comes to forecasting the day-to-day volatility of the stock market, can modern machine learning actually beat the classical models that finance has relied on for decades? Let me walk you through what I built and what I found.

Slide 2: Introduction (about 50 seconds)

First, the setting. Volatility is the size of an asset's daily price swings, and it is the central quantity of financial risk management: it prices options, it sets value-at-risk limits and margins, and it determines how large a position a trader may take. There is a famous asymmetry in financial data. The direction of tomorrow's return is essentially unpredictable. But its size is not: markets move through calm spells and turbulent spells, and that persistence means volatility can be forecast. Classical econometrics has exploited this for forty years with the GARCH family. My question is whether XGBoost, a gradient boosting model, and an LSTM neural network can do better, when every model is tested fairly, inside one harness, with no look-ahead bias, and with a proper significance test at the end.

Slide 3: Why Volatility Can Be Forecast (about 45 seconds)

This slide is the empirical foundation of the whole project. On the left, the autocorrelation function of raw returns: essentially flat. Past returns tell you nothing about the direction of the next one, which is exactly why I am not trying to predict returns. On the right, the same calculation on absolute returns: the autocorrelation is strong, and it decays slowly across dozens of lags. Big days cluster with big days, calm days with calm days. This is volatility clustering, and this persistence is the one signal that every model in this study feeds on.

Slide 4: Objectives (about 45 seconds)

Six objectives. One, build honest classical baselines: naive, rolling, exponentially weighted, and the GARCH family, including the leverage variants GJR-GARCH and EGARCH. Two, build a leak-free walk-forward evaluation, scored with RMSE, MAE, QLIKE, and the Diebold–Mariano significance test. Three, build a walk-forward XGBoost with justified features. Four, build a carefully regularized LSTM with explicit anti-leakage guards. Five, compare all the models on one shared test window and check which differences are statistically real. And six, deliver a reproducible, modular codebase, which is now public on GitHub.

Slide 5: Dataset (about 50 seconds)

The data. Daily closes of the S&P 500 from Yahoo Finance, from January 2008 to the end of June 2026. That is 4,650 trading days, about eighteen and a half years, converted to log returns. The preprocessing is minimal: log-difference the adjusted closes and drop the first missing value. The last thirty percent, 1,395 days from December 2020 to June 2026, is held out as the test window, and every model is scored on exactly those days. Note what that window contains: the 2022 bear market and the sharp tariff shock of April 2025. So the models are stress-tested out of sample on two genuine crashes, while the COVID crash of 2020 sits in the training data.

Slide 6: Tools and Technologies (about 40 seconds)

The stack is standard Python. Pandas, NumPy and yfinance for the data; the arch package for maximum-likelihood GARCH estimation; XGBoost for the trees; PyTorch for the LSTM; statsmodels for the diagnostics. Everything lives in a small modular package called volforecast, developed in Jupyter and VS Code, versioned with git, with the report written in LaTeX. One engineering detail worth mentioning: the five expensive model fits are mutually independent, so they run in parallel processes, and the entire study reproduces end to end in about one minute.

Slide 7: The Models (about 55 seconds)

The contenders. On the classical side: the naive forecast, which is simply yesterday's squared return; a twenty-one day rolling variance; the RiskMetrics exponentially weighted average; and the GARCH family. Plain GARCH says tomorrow's variance is a constant, plus alpha times yesterday's squared return, plus beta times yesterday's variance. GJR-GARCH and EGARCH add the leverage effect, the fact that a fall raises future volatility more than a rise of the same size. On the learned side: XGBoost, refit monthly inside the walk-forward, and an LSTM that reads the last twenty days of features. Both see identical inputs, recent returns and realized variances, and predict the same target, a smoothed forward-looking variance, with a five-day buffer so no future information ever leaks into training.

Slide 8: Evaluation (about 50 seconds)

How they are judged. The golden rule is no look-ahead: the forecast for any day uses only information up to the previous day, and models are re-estimated about monthly as the window advances. The headline metric is QLIKE, for two reasons. Its ranking of models is robust to the noise in the squared-return proxy, and it punishes under-predicting variance more than over-predicting it, which is exactly the mistake a risk manager fears most. RMSE and MAE are reported as well, for point accuracy. And every gap on the

leaderboard is checked with the Diebold–Mariano test, so a difference is only called real when the statistics support it.

Slide 9: Results (about 55 seconds)

The results. EGARCH wins, with a QLIKE of 1.504, and GJR-GARCH sits just behind it. The striking entry is third place: the LSTM, ahead of plain GARCH. Look at the point-accuracy columns too: the LSTM posts the best RMSE of all eight models, and XGBoost the best MAE. The naive baseline's QLIKE of over five thousand is not a bug; the loss explodes whenever a near-zero forecast meets a real market move, which is precisely why that baseline is inadequate for risk. On the right, all the forecasts track the same volatility clusters, with the April 2025 spike dominating the window; the models differ mainly in how quickly they react to it.

Slide 10: Is the Winner Really Better? (about 50 seconds)

But is the winner really better? Three Diebold–Mariano results. EGARCH against GJR-GARCH: p equals 0.47, so the two leverage models are statistically tied and effectively interchangeable. EGARCH against plain GARCH: p below ten to the minus four, decisively significant; on this window the leverage effect genuinely matters. And EGARCH against the LSTM: p equals 0.19, not significant, so the neural network is statistically indistinguishable from the winner. The chart shows the winning EGARCH forecast hugging realized volatility, including through the 2025 spike.

Slide 11: Why Do the Leverage Models Win? (about 55 seconds)

Why do the leverage models win? Because the test window is dominated by two sharp crashes, and the leverage term converts a large fall directly into higher next-day variance; that is exactly the mechanism these models encode. And why does the classical family as a whole stay ahead of machine learning? Three structural reasons. The GARCH models are fit by maximum likelihood on the correct variance objective, while the learned models train on a smoothed substitute target. Roughly three thousand daily observations is small data, which favours a four-parameter recursion over a neural network. And QLIKE rewards reacting to a shock the very next day, which the recursion does by construction. The honest limitations: one asset, one window, a noisy proxy, and deliberately modest tuning.

Slide 12: Conclusion and Takeaways (about 50 seconds)

So, can machine learning beat the classical models? On this evidence, no: a monthly re-estimated EGARCH gives the best risk-relevant forecasts, and no learned model outranks the leverage GARCH family. But the answer is narrower than it once was. The LSTM finished third, ahead of plain GARCH, statistically tied with the winner, and the learned models took both point-accuracy metrics. The main lesson of the project is methodological: the leak-free walk-forward harness and the choice of metric mattered more than any modelling decision. Everything you have seen, the code, the notebooks, the report and the figures, is public at github.com/iitgF/T9 and reproduces in about one minute. Thank you for watching.